



HABLAR IA

Irene Gómez Picazo

Carme Ferrer Alòs

Diego Cambeiro Ruiz

Raquel Guerricabeytia López

CURSO ACADÉMICO 2026-2027
E.T.S.I. DE TELECOMUNICACIÓN
UNIVERSITAT POLITÈCNICA DE VALÈNCIA
7 OCTUBRE 2026

ÍNDICE

1. Introducción	3
2. Objetivos	3
3. Preparación del entorno.....	4
3.1. Comprobación del entorno	4
3.2. Ejecución del backend del IDE.....	4
3.3. Ejecución de la interfaz gráfica.....	5
4. Desarrollo del agente Hyperion	5
4.1. Clonación y configuración del proyecto.....	6
4.2. Implementación del sistema RAG, memoria y modelo de lenguaje	6
4.3. Implementación de herramientas (Tool Calling) e interacción con el IDE	6
4.4. Guardrails (Seguridad y Contención).....	7
5. Empaquetado y Despliegue en Docker.....	8
5.1. Construcción de la Imagen Local	8
5.2. Etiquetado y Publicación	8
6. Conclusión	9

1. Introducción

Este proyecto consiste en desarrollar un microservicio agente denominado Hyperion, cuyo objetivo es permitir la interacción con el entorno HYPER-AI IDE mediante lenguaje natural. El agente actúa como intermediario entre el usuario y el entorno de desarrollo, permitiendo interpretar las instrucciones recibidas y proporcionar respuestas utilizando un modelo de lenguaje. Además, el sistema está diseñado para incorporar diferentes capacidades de inteligencia artificial, entre las que destacan la recuperación de información mediante Retrieval-Augmented Generation (RAG), la memoria de las conversaciones y la ejecución de acciones sobre el propio IDE.

La arquitectura propuesta contempla también mecanismos de seguridad mediante guardrails, con el objetivo de rechazar consultas que no estén relacionadas con la finalidad del agente, así como la posibilidad de solicitar confirmación antes de ejecutar determinadas acciones destructivas. La comunicación con el agente se realiza mediante un endpoint HTTP POST/chat, expuesto en el puerto 8000, utilizando Server-Sent Events (SSE) para proporcionar las respuestas de forma progresiva.

Como punto de partida se utiliza el repositorio hyperion-starter, sobre el que se van incorporando de forma incremental las funcionalidades requeridas. El desarrollo se realiza junto con los componentes proporcionados del HYPER-AI IDE, concretamente ide-backend e ide-gui.

2. Objetivos

El objetivo principal de la práctica es desarrollar e integrar un agente conversacional capaz de interactuar con el entorno HYPER-AI IDE mediante lenguaje natural. Para alcanzar este objetivo general, se plantean los siguientes objetivos específicos:

- Implementar un microservicio accesible mediante el endpoint POST /chat, utilizando comunicación mediante Server-Sent Events (SSE).
- Integrar un modelo de lenguaje capaz de procesar las consultas del usuario y generar respuestas de forma progresiva.

- Incorporar un sistema de memoria de sesión asociada a un `user_id`, permitiendo mantener el contexto de las conversaciones.
- Implementar un mecanismo de RAG que permita consultar la documentación de HYPER-AI y utilizar la información recuperada para responder a las preguntas del usuario.
- Dotar al agente de herramientas que permitan realizar acciones sobre el IDE, como crear, editar, abrir y eliminar archivos.
- Incorporar guardrails que permitan detectar y rechazar consultas que no sean relevantes para la finalidad del agente.
- Construir y desplegar el servicio mediante Docker, incluyendo la generación de una imagen del agente y su publicación en Docker Hub.

3. Preparación del entorno

Antes de comenzar con el desarrollo del agente Hyperion, fue necesario preparar el entorno de ejecución proporcionado por HYPER-AI. Para ello hemos utilizado los contenedores Docker correspondientes al backend y a la interfaz gráfica del IDE.

3.1. Comprobación del entorno

Para empezar, hemos comprobado que el motor de Docker estuviese en ejecución. También hemos verificado la disponibilidad de los puertos para la ejecución de los servicios: 3001 para el backend del IDE, 5000 para la interfaz gráfica y 8000 para el futuro servicio Hyperion.

3.2. Ejecución del backend del IDE

En primer lugar, hemos puesto en funcionamiento `ide-backend`, encargado de proporcionar la API de archivos utilizada por el entorno de desarrollo. Para iniciar sesión hemos utilizado el comando:

```
docker run --rm -p 3001:3001 -e AUTH_ENABLED=false --name ide-backend donmichael/ide-backend:latest
```

También comprobamos que la autenticación estaba deshabilitada para este entorno de desarrollo mediante `AUTH_ENABLED=false`.

3.3. Ejecución de la interfaz gráfica

Una vez iniciado el backend, ejecutamos el contenedor ide-gui, correspondiente a la interfaz web del HYPER-AI IDE. Utilizamos el siguiente comando para ello:

```
docker run --rm -p 5000:80 --name ide-gui donmichael/ide-gui:latest
```

Este contenedor permite acceder al entorno de desarrollo mediante un navegador web utilizando la dirección <http://localhost:5000/>.

3.4. Comprobación del entorno

La descarga y ejecución correcta de las imágenes base se verificó mediante el comando `docker images`, confirmando la disponibilidad de los contenedores iniciales.

La descarga y ejecución correcta de las imágenes base se verificó mediante el comando `docker images`, confirmando la disponibilidad de los contenedores iniciales.

```
C:\Users\carne>docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
donmichael/ide-backend:latest	caa8f470280e	672MB	175MB	U
donmichael/ide-gui:latest	7de924f5e223	74.1MB	21.1MB	U

Comprobación de las imágenes base del IDE (backend y gui) cargadas en Docker.

La interfaz gráfica se utiliza desde <http://localhost:5000/>, mientras que el backend funciona en <http://localhost:3001>. Esta configuración constituye la base sobre la que posteriormente se integra el agente Hyperion, que será ejecutado en el puerto 8000.

4. Desarrollo del agente Hyperion

Una vez preparado el entorno para la ejecución del HYPER-AI IDE, procedimos al desarrollo del agente Hyperion a partir del repositorio inicial ya proporcionado (hyperion-starter).

4.1. Clonación y configuración del proyecto

Como punto de partida para el desarrollo, clonamos el repositorio hyperion-starter y accedimos desde una terminal a la carpeta. Antes de comenzar a modificar la lógica principal del agente, se comprobó el entorno de ejecución de Python y la disponibilidad de uv, herramienta utilizada para ejecutar y gestionar el proyecto.

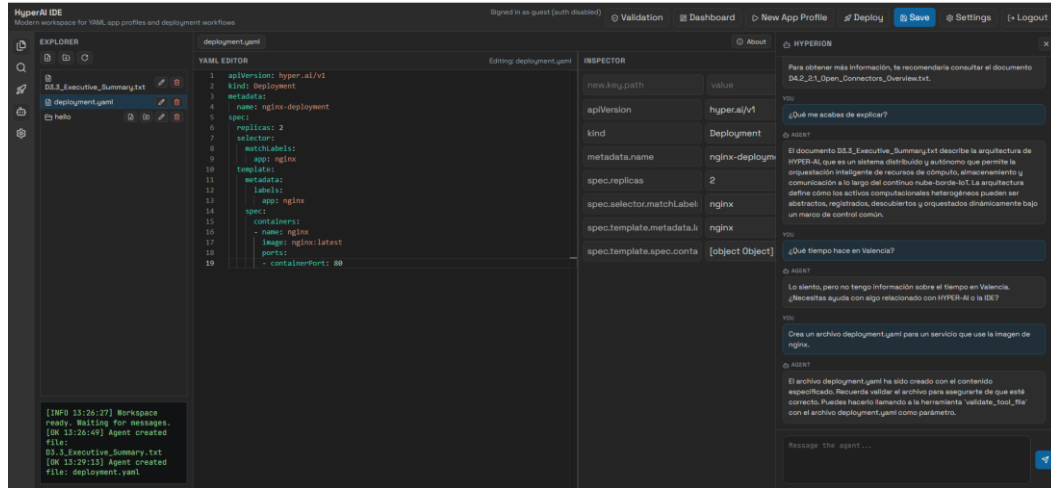
4.2. Implementación del sistema RAG, memoria y modelo de lenguaje

Una de las funcionalidades principales requeridas para el agente Hyperion es la capacidad de consultar la documentación de HYPER-AI para proporcionar respuestas fundamentadas en información relevante. Para ello se incorporó un sistema RAG (Retrieval-Augmented Generation) que, ante cada consulta, recupera los tres fragmentos de documentación con mayor similitud respecto a la pregunta del usuario. Este contexto recuperado se incorpora al System Prompt. La memoria de conversación se implementa mediante un diccionario en el que cada `user_id` actúa como identificador de una sesión independiente. Para evitar que el historial crezca indefinidamente, se establece un límite de diez mensajes mediante la constante `MAX_HISTORY`. El modelo se configura mediante ChatOpenAI, utilizando el servidor LLM proporcionado (modelo llama3.1 con temperatura 0 para mayor precisión).

4.3. Implementación de herramientas (Tool Calling) e interacción con el IDE

Se creó el archivo `tools.py`, donde se definen las herramientas que el modelo puede utilizar: `create_folder`, `delete_folder`, `create_file`, `edit_file`, `delete_file`, `read_tool_file` y `validate_tool_file`. Las herramientas se registran mediante el decorador `@tool` de LangChain. Como medida de control, se implementó una función encargada de comprobar que las rutas utilizadas por las herramientas sean relativas al workspace, rechazando rutas absolutas o con ... El modelo evalúa la necesidad de herramientas de forma nativa o extrayendo estructuras JSON

del texto, reinyectando el resultado en el contexto para generar la respuesta final mediante iteraciones controladas (MAX_ROUNDS).



Creación automatizada del archivo deployment.yaml en el explorador del IDE mediante el uso de herramientas (Tool Calling) por parte del agente.

4.4. Guardrails (Seguridad y Contención)

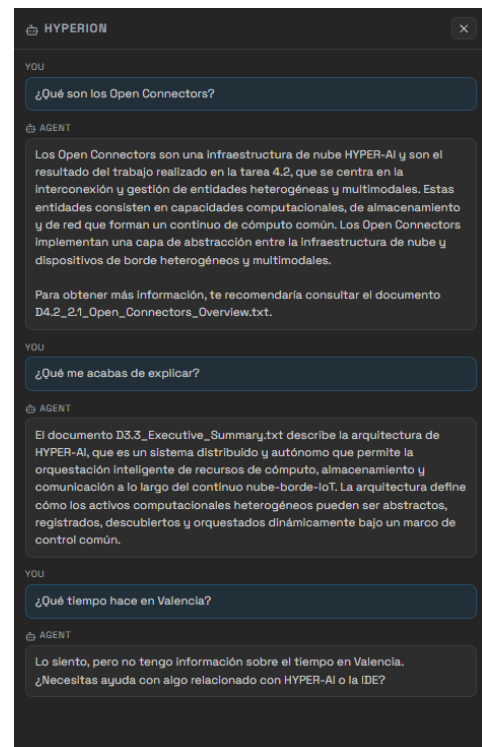
Para garantizar que el agente Hyperion opere estrictamente dentro de los límites del dominio de HYPER-AI y la manipulación del IDE, se ha implementado un sistema de guardrails estructurado en dos capas defensivas:

- **Capa de Aplicación (Keyword Filtering):**

Intercepta consultas en main.py antes de llamar al LLM. Si la pregunta carece de vocabulario de dominio (ej. deploy, yaml, IDE), se rechaza automáticamente mediante SSE.

- **Capa de Modelo (System Prompting):**

Directrices estrictas instruyen al modelo a fundamentar sus respuestas exclusivamente en el contexto recuperado por el RAG y no inventar información sobre el proyecto HYPER-AI.



Las pruebas realizadas confirmaron la robustez de esta arquitectura. Consultas fuera de dominio ("¿Qué tiempo hace en Valencia?") son interceptadas y rechazadas correctamente, preservando el propósito funcional del agente.

5. Empaquetado y Despliegue en Docker

La fase final consistió en encapsular la lógica del agente en una imagen Docker independiente y publicarla para su evaluación.

5.1. Construcción de la Imagen Local

Mediante el archivo Dockerfile y docker-compose.yaml, se configuró un entorno basado en Python que instala las dependencias mediante uv y expone el puerto 8000. La construcción local (docker compose up --build) generó con éxito la imagen hyperion-starter-hyperion:latest con un tamaño aproximado de 3.19 GB.

```
C:\Users\carne\Documents\Github-Karne\hackathon-veles-hack\hyperion-starter>docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
donnmichael/ide-backend:latest	caa8f478280e	672MB	175MB	U
donnmichael/ide-gui:latest	7de924f5e223	74.1MB	21.1MB	U
hyperion-starter-hyperion:latest	c135b872b247	3.19GB	3.19GB	

Construcción exitosa de la imagen local del agente Hyperion.

5.2. Etiquetado y Publicación

Para cumplir con los requisitos de entrega, la imagen local se etiquetó vinculándola a una cuenta de Docker Hub (cferalo). El proceso se completó con los siguientes comandos tras iniciar sesión:

- `docker tag hyperion-starter-hyperion cferalo/hyperion:latest`
- `docker push cferalo/hyperion:latest`

El comando `docker images` verificó la creación exitosa del alias `cferalo/hyperion:latest`, listo para distribución.

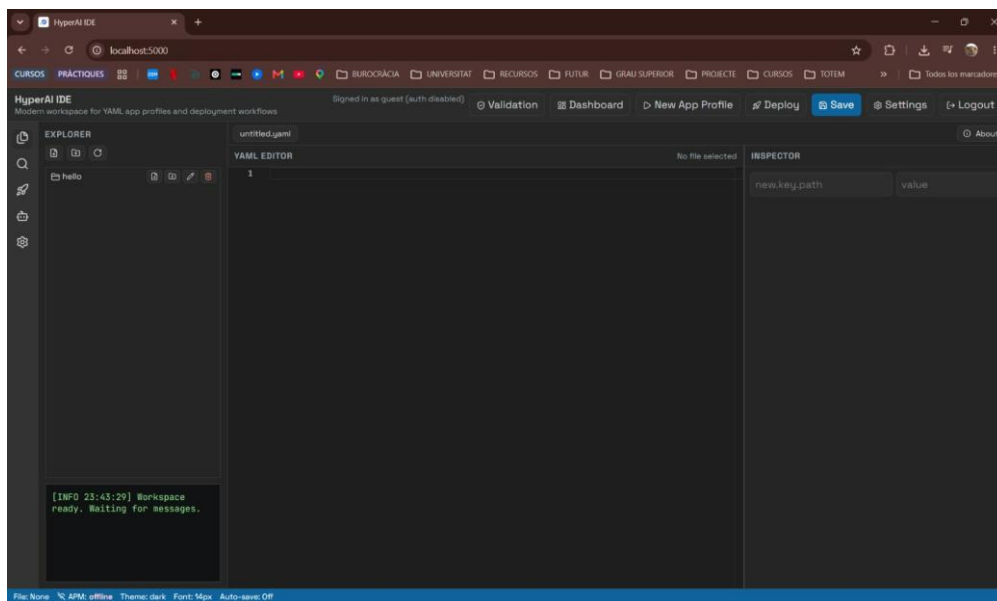
```
C:\Users\carne\Documents\Github-Karne\hackathon-veles-hack\hyperion-starter>docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
cferalo/hyperion:latest	a14bf14560ec	9.32GB	3.19GB	
donnmichael/ide-backend:latest	caa8f478280e	672MB	175MB	U
donnmichael/ide-gui:latest	7de924f5e223	74.1MB	21.1MB	U
hyperion-starter-hyperion:latest	c135b872b247	9.32GB	3.19GB	U

Etiquetado de la imagen vinculada al repositorio de Docker Hub del usuario (cferalo).

5.3. Validación Final del Despliegue

Una vez levantado el contenedor Hyperion junto con los servicios base del IDE, se verificó la correcta comunicación entre componentes. La interfaz gráfica (localhost:5000) cargó el panel de control del asistente, confirmando que el *workspace* estaba inicializado y esperando mensajes de forma estable.



Interfaz final del IDE operativo y conectado a la red local esperando interacciones.

6. Conclusión

Como conclusión, a pesar del nivel formativo que teníamos dentro del equipo y las limitaciones de tiempo del equipo, ha sido un proyecto que nos ha aportado mucho en cuanto a experiencia académica.

Desde el principio al ser un grupo de 4, con disponibilidades variadas, hemos desarrollado del mismo proyecto dos versiones diferentes. Esto ha permitido el reparto de tareas y una exploración más profunda de las propuestas.

Para nosotros, ha sido una oportunidad única, ya que es la primera vez que participamos en una experiencia de este estilo. Estamos muy agradecidos y con ganas de participar en el siguiente.